REVERSE ENGINEERING

NUL Cyber Squad • Training Notes

INTRODUCTION

🔍 Understanding Reverse Engineering

Reverse Engineering (RE) is the process of deconstructing a compiled program to understand its logic, discover hidden features, or find vulnerabilities. In Capture The Flag (CTF) competitions, the goal is typically to extract a hidden **flag** or a **password**.

PREPARATION

🐧 Running Binaries on Linux

Before analyzing, you must identify the file type and ensure it has execution permissions.

```
# Check file architecture (e.g., ELF 64-bit)
$ file challenge

# Make the file executable
$ chmod +x challenge

# Run the program
$ ./challenge
```

TOOLS

🛠️ The Basic Toolkit

- **strings:** Searches for printable text inside the binary. Often finds flags or passwords immediately.
- **file:** Tells you what kind of file it is (executable, script, etc.).
- **ltrace:** Traces **library calls** (like `strcmp`) to see how the program handles your input.
- **strace:** Traces **system calls** (like open or `read`) to see how it interacts with the OS.

```
# Example: Using ltrace to see a password check
$ ltrace ./challenge
strcmp("your_input", "SUPER_SECRET_FLAG") = -1
```

ADVANCED

🧠 Ghidra: Decompilation

When simple tools fail, use **Ghidra**. It converts machine code (assembly) back into readable C-like code. This allows you to follow the program's logic, see if-statements, and understand complex math or encryption.

METHODOLOGY

🧪 Simple CTF Workflow

1. **file:** Identify the architecture.
2. **strings:** Look for hardcoded flags.
3. **ltrace:** Observe library functions and comparisons.
4. **Ghidra:** Perform a deep-dive analysis of the logic.

DEFEND • PROTECT • EMPOWER